

Future-proof submdspan_mapping

Document #: D3663R2
Date: 2025-06-30
Project: Programming Language C++
LEWG
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>

Contents

- 1 Author
- 2 Revision history
- 3 Abstract
- 4 Motivation
 - 4.1 Summary
 - 4.2 Four categories of slice types
 - 4.3 Why undefined behavior?
 - 4.4 Set of contiguous subrange slices was expanded once and may be again
 - 4.5 Other existing categories could be expanded
 - 4.6 WG21 may add a new slice category
- 5 Solution: Make submdspan canonicalize slices
 - 5.1 Summary
 - 5.2 Canonicalization rules
 - 5.3 How to apply canonicalization
 - 5.4 Canonicalization is reusable
 - 5.5 What about precondition checking?
 - 5.6 Why implementers should do this anyway
- 6 Performance
 - 6.1 Speculation on compile-time and run-time effects
 - 6.2 Hypothetical performance mitigations
 - 6.3 submdspan can skip canonicalization for Standard layout mappings

- 7 Implementation
 - 7.1 Configuration and build
 - 7.2 Benchmark
 - 7.2.1 Benchmark code details
 - 7.2.2 Preliminary results
 - 7.2.3 Preliminary conclusions
- 8 Acknowledgements
- 9 Proposed wording
 - 9.1 Increment `__cpp_lib_submdspan` feature test macro
 - 9.2 Change `[mdspan.syn]`
 - 9.3 Change `[mdspan.sub.helpers]`
 - 9.4 Add section `[mdspan.sub.slices]`, “Slices definitions and canonicalization”
 - 9.5 Change section `[mdspan.sub.extents]`
 - 9.6 Requirements of all `submdspan_mapping` customizations
 - 9.7 Change `[mdspan.sub.map.common]`
 - 9.8 `submdspan` function template

§ 1 Author

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)

§ 2 Revision history

- Revision 0 to be submitted 2025-03-17
 - 2025-04-15: Reviewed by LEWG and forwarded to LWG.
- Revision 1 to be submitted 2025-05-19 for the pre-Sofia mailing
 - Add Mandates and Preconditions to *canonical-ice* that the input be representable as a value of type `IndexType`.
 - Fix *subtract-ice* to use `constant_wrapper` arithmetic: that is, apply *canonical-ice* first, then subtract the resulting `constant_wrapper` objects.
 - Use structured binding for pair slices, not `get`; reorder Mandates and Returns clauses to put the structured binding part last. (For Mandates, this means that implementations need only try the structured binding; if it fails, it’s ill-formed.)

- submdspan can't actually be constrained on the specific `submdspan_mapping(input.mapping(), slices...)` call being well-formed, because there's no way to constrain on a structured binding declaration being well formed. (Structured binding is a declaration, while constraints require expressions.) Instead, change the constraint on `submdspan` to be that `submdspan_mapping(input.mapping(), full_extent, ..., full_extent)` is well-formed (call that an exposition-only concept *mapping-sliceable-with-full-extents*), and make it just a Mandate that `submdspan_mapping(input.mapping(), slices...)` be well-formed.
- Add a new section for requirements on sliceable mappings, with a new named requirement “sliceable layout mapping,” and an exception for the standard mappings. The requirements are currently expressed in parts of `[mdspan.sub.map.common]` and `[mdspan.sub.sub]` (e.g., 5.2 and 5.3, with Note 2 after 5.3).
- Add new function `submdspan_canonicalize_slices` to `<mdspan>` header synopsis.
- Fix non-wording Abstract and the rest of the non-wording sections to say that we do actually propose these changes.
- Add “Canonicalization is reusable” non-wording section to explain why `submdspan_canonicalize_mapping` is not exposition-only.
- Update P2781 (`std::constant_wrapper`) references from R7 to R8.
- Add an Editorial Note to explain that the standard layout mappings could accept all valid slice types, instead of just the canonical ones. This should alleviate concerns about potential performance effects of canonicalization. However, retain (in Editorial Notes) suggested wording for the pass-through, just in case.
- Define an exposition-only function *check-static-bounds* to avoid repetition when stating Mandates on canonical and valid slice types.
- Revision 2 to be submitted 2025-07-15 for the post-Sofia mailing
 - Add new implementation link.
 - Based on implementation experience with *canonical-ice*,
 - specify its return value using `cw` instead of `constant_wrapper` (to avoid issues with the first template parameter of `constant_wrapper` actually being of an exposition-only type instead of the expected `IndexType`), and
 - cast the result of *index-cast* to `IndexType` before using it as the constant value in `cw`.
 - Based on implementation experience with *check-static-bounds*,

- rewrite *check-static-bounds* wording to use only $S_k\{\}$ instead of s_k , so that all of the expressions in the wording are valid constant expressions;
- specify *check-static-bounds* to take only one slice type as an explicit template parameter, instead of taking all the slices as a parameter pack (as taking the slices as function parameters would make calling *check-static-bounds* from a function taking slice parameter(s) not a constant expression); and
- check wording of *check-static-bounds* so that it only assumes that types are default constructible in constant expressions if they are *integral-constant-like*.
- Based on implementation experience with `submdspan`, in `submdspan`’s “*Effects: Equivalent to*” code,
 - fix formatting of `canonical_slices`,
 - add drive-by fix for `src.data_handle()` (original was `src.data()`), and
 - add drive-by fix adding missing `typename` before `AccessorPolicy::offset_policy`.
- Based on LWG reflector discussion on 2025/06/02,
 - fix *first_* and *last_* wording to say e.g., “ s_{k0} produced by `auto [s_{k0}, _] = s_k`,” instead of “ s_{k1} if the structured binding declaration `auto [s_{k0}, s_{k1}] = s_k`; is well-formed”;
 - fix *check-static-bounds Returns* clause to follow the above pattern instance of saying “if the structured binding declaration `auto [s_{k0}, s_{k1}] = declval<Slice>()`; is well-formed...”;
 - adjust *check-static-bounds Returns* clause to make the function well-formed in cases where the input slice is one of the three valid non-pair-like slice types but would otherwise return unknown; and
 - adjust `submdspan_canonicalize_slices Returns` clause so that the last (pair-like) case follows the same pattern as *first_* and *last_*.
- Ensure that *canonical-ice* returns either `cw<IndexType(Value)>` or `IndexType(Value)` for the value `Value`.
- Expand discussion of implementation, and add preliminary benchmark results.

§ 3 Abstract

Currently, the `submdspan` function can call a `submdspan_mapping` customization with any valid slice type. This means that `submdspan_mapping` customizations may be ill-formed, possibly even without a diagnostic, if future C++ versions expand the set of valid slice types. This will happen if [P2769R3](#) is merged into the Working Draft for C++29, as that would generalize *tuple-like* from a concrete list of types to a concept. It may also happen in the future for other categories of slice types.

We propose fixing this with a “canonicalization” approach. First, define a fixed set of “canonical” slice types that represent all valid slices, and a function `submdspan_canonicalize_slices` to map valid slices to their canonical versions. Then, change `submdspan` so it always canonicalizes its input slices, and so it only ever calls `submdspan_mapping` customizations with canonical slices.

§ 4 Motivation

§ 4.1 Summary

The current `submdspan` wording has the following constraint ([\[mdspan.sub.sub\] 3.2](#)).

... the expression `submdspan_mapping(src.mapping(), slices...)` is well-formed when treated as an unevaluated operand.

However, nothing currently requires `submdspan_mapping` to be ill-formed when any of its slices is *not* a valid slice type. Thus, the `submdspan` function can call a `submdspan_mapping` customization with any slice type that is valid for `submdspan`. This means that the following scenario invokes undefined behavior.

1. A user defines a layout mapping `my_layout::mapping` with a C++26 - compatible `submdspan_mapping` customization.
2. A later C++ version adds a new valid slice type `new_slice`.
3. `submdspan` is called with `my_layout::mapping` and a `new_slice` slice.

This is true whether the new slice type is in one of the existing four categories of slice types or is in a new category. It is a scenario that has already occurred and may occur again, as we will explain below.

§ 4.2 Four categories of slice types

The Working Draft has four categories of slice types. Let `S` be an input slice type of `submdspan`.

1. “Full”: `S` is convertible to `full_extent_t`. This means “all of the indices in that extent.”

2. “Integer”: *S* is convertible to (the layout mapping’s) `index_type`. This means “fix the slice to view that extent at only that index.” Each integer slice reduces the result’s rank by one.
3. “Contiguous subrange”: *S* models *index-pair-like*`<index_type>`, or *S* is a `strided_slice` specialization whose `stride_type` is *integral-constant-like* with value 1. A contiguous subrange slice represents a contiguous range of indices in that extent. The fact that the indices are contiguous (in other words, that they have stride 1) is known at compile time as a function of the slice’s type alone. The special case of `strided_slice` with compile-time stride 1 is the only way to represent a contiguous index range with run-time offset but compile-time extent. (See [P3355r2](#), which was voted into the Working Draft for C++26 at the 2024 Wrocław meeting.)
4. “Strided”: *S* is a specialization of `strided_slice`, but is not a contiguous subrange. It may represent a noncontiguous set of indices in that extent.

The current Working Draft uses the term *unit-stride slice* (defined in [P3355](#)) to include both full and contiguous subrange slices.

§ 4.3 Why undefined behavior?

Why would calling a C++26 - compatible `submdspan_mapping` customization with a post - C++26 slice type be undefined behavior? Why wouldn’t it just be ill-formed? The problem is that the customization might actually be well-formed for new slice types, but it might do something that `submdspan` doesn’t expect. For example, it might violate preconditions of the `submdspan_mapping` customization, or it might produce a `submdspan_mapping_result` that violates postconditions of `submdspan` (e.g., views the wrong elements). This could happen for two reasons.

1. The user has overloaded the customization incorrectly. It works correctly for all C++26 slice types, but incorrectly maps the new slice type to one of the four C++26 cases.
2. The user intends for the customization to be well-formed with invalid slice types. It might have behavior that makes sense for the user, but not for `submdspan`, as in the following example.

```
struct my_tag_slice_t {
    template<size_t k>
    friend constexpr size_t get() const {
        if constexpr (k == 0) {
            return 42;
        }
        else if constexpr (k == 1) {
            return 100;
        }
        else {
            static_assert(false);
        }
    }
};
```

```

    }
}
};

namespace std {
    struct tuple_size<my_tag_slice_t>
        : integral_constant<size_t, 2> {};
    template<size_t I>
    struct tuple_element<I, my_tag_slice_t> {
        using type = size_t;
    };
} // namespace std

template<class... Slices>
friend constexpr auto
submdspan_mapping(const mapping& src, Slices... slices)
{
    if constexpr ((valid_cpp26_slice<Slices> && ...)) {
        // ... all slices are valid C++26 slices,
        // so do the normal C++26 thing ...
    }
    else if constexpr (is_same_v<Slices, my_tag_slice_t> && ...) {
        // my_tag_slice_t is not a valid C++26 slice,
        // but it has an unambiguous interpretation
        // as a pair of indices.
        return std::string("Hello, I am doing something weird");
    }
    else {
        static_assert(false);
    }
}
}

```

§ 4.4 Set of contiguous subrange slices was expanded once and may be again

WG21 has already expanded the set of contiguous subrange slice types once, and is likely to do so again. This is because it depends on the exposition-only concept *pair-like*. Adoption of [P2819R2](#) into the Working Draft for C++26 at the 2024 Tokyo meeting made complex a *pair-like* type, and therefore a valid slice type for `submdspan`. The *pair-like* concept depends in turn on the exposition-only concept *tuple-like*. [P2769R3](#), currently in LEWG review, proposes generalizing *tuple-like* from a concrete list of types to a concept. That would make user-defined pair types valid slice types.

Before adoption of [P2819R2](#), a user-defined layout mapping's `submdspan_mapping` customization could determine whether a type is *T* is *pair-like* by testing exhaustively whether it is `array<X, 2>` for some *X*, `tuple<X, Y>` or `pair<X, Y>` for some *X* and *Y*, or `ranges::subrange` with `tuple_size` 2. P2819 would break a customization that uses this approach. Similarly, adoption of [P2769R3](#) would break customizations that add `complex<R>` (for some *R*) to this list of types to test exhaustively.

It would be reasonable for `submdspan` to generalize *pair-like* into any type for which structured binding into two elements is well-formed. If the two elements are both convertible to `index_type`, then this has an unambiguous interpretation as a contiguous subrange. Thus, users should write their customizations generically to that interface. However, in terms of the Standard, there are two issues with that approach.

1. Standard Library implementers need to assume that users read the Standard as narrowly as possible. C++26 does not forbid calling a `submdspan_mapping` customization with a user-defined pair type, but it also does not restrict its behavior in that case.
2. This approach would not help if other slice categories are expanded, or if a new slice category is added, as the following sections explain.

§ 4.5 Other existing categories could be expanded

The only types currently in the “strided” category are specializations of `strided_slice`. Future versions of C++ might expand this category. In our view, this would have little benefit. Nevertheless, nothing stops WG21 from doing so. For example, C++29 might (hypothetically) make a strided slice any type that meets the following exposition-only *strided-slice*<`index_type`> concept.

```
template<class T>
concept integral_not_bool = std::is_integral_v<T> &&
    ! std::is_same_v<T, bool>;

template<class IndexType, class T>
concept strided_slice = semiregular<T> &&
    convertible_to<T::offset_type, IndexType> &&
    convertible_to<T::extent_type, IndexType> &&
    convertible_to<T::stride_type, IndexType> &&
    requires(T t) {
        { t.offset } -> convertible_to<IndexType>;
        { t.extent } -> convertible_to<IndexType>;
        { t.stride } -> convertible_to<IndexType>;
    };

```

The following user-defined type `my_slice` would meet *strided-slice*<`size_t`>, even though

- it is not convertible to or from `strided_slice<size_t, size_t, size_t>`;
- it has a base class, which `strided_slice` specializations cannot, per [mdspan.sub.strided.slice] 2; and
- it has members other than `offset`, `extent`, and `stride`, which `strided_slice` cannot, per [mdspan.sub.strided.slice] 2.

```
struct my_base {};

struct my_slice : public my_base {

```



```

using offset_type = size_t;
using extent_type = size_t;
using stride_type = size_t;

offset_type offset;
extent_type extent;
stride_type stride;

// User-declared constructor makes this not an aggregate
my_slice(extent_type ext) : offset(0), extent(ext), stride(1) {}

// strided_slice is not allowed to have extra members
std::string label = "my favorite slice label";
std::vector<int, 3> member_function() const { return {1, 2, 3};
}

// my_slice is not convertible to or from strided_slice
template<class Offset, class Extent, class Stride>
my_slice(strided_slice<Offset, Extent, Stride>) = delete;

template<class Offset, class Extent, class Stride>
operator strided_slice<Offset, Extent, Stride>() const = delete;
};

```

Now suppose that a user has defined a `submdspan_mapping` customization in a C++26-compatible way for their custom layout mapping `custom_layout::mapping`. Then, `my_slice` likely would not work for this layout mapping in C++29, even though the hypothetical `_strided_slice_` concept makes it possible to interpret `my_slice` unambiguously as a strided slice, and to write generic code that does so.

Unlike with contiguous subrange slices, users have no way to anticipate all possible ways that future Standard versions might expand the set of valid strided slice types. For example, C++29 could decide that strided slices must be aggregate types (which would exclude the above `my_base`), or that any type for which a structured binding into three elements is well-formed is a valid strided slice. Furthermore, `my_slice` with `stride = cw<1, size_t>` should reasonably count as a contiguous subrange slice type; expanding one set risks conflicts between sets.

§ 4.6 WG21 may add a new slice category

The same problem would arise if future versions of C++ introduce a new category of slices. For example, suppose that C++29 hypothetically adds an “indirect slice” category that takes an arbitrary tuple of indices to include in that extent of the slice. The Mandates of `submdspan` ([`mdspan.sub.sub`] 4.3) make it ill-formed to call with a new kind of slice, so this hypothetical C++29 addition would need to add a new clause there. That would not be a breaking change in itself. The problem is that nothing currently requires `submdspan_mapping` to be ill-formed when any of its slices is not a valid slice type.

§ 5 Solution: Make submdspan canonicalize slices

§ 5.1 Summary

There is a tension between submdspan users and custom mapping authors. Users of submdspan want it to work with all kinds of slice types. For example, they might want to write their own tuple types that are standard-layout if all their template parameters are (unlike `std::tuple`), and would expect submdspan to work with them. In contrast, users who customize submdspan_mapping for their own layout mappings want a “future-proof” interface. They only want submdspan to call submdspan_mapping for a known and unchanging set of types.

We propose resolving this tension by changing submdspan so that it “canonicalizes” its slice inputs before passing them to submdspan_mapping. The submdspan function still takes any valid slice types, but it maps each input slice type to a “canonical” slice type according to rules that we will explain below. As a result, a submdspan_mapping customization would ever be called with the following concrete list of slice types.

1. `full_extent_t`;
2. `index_type` (that is, the layout mapping’s `index_type`);
3. `constant_wrapper<Value, index_type>` for some `Value`; or
4. `strided_slice` where each member is either `index_type` or `constant_wrapper<Value, index_type>` for some `Value`.

The `constant_wrapper` class template comes from [P2781](#). LEWG design-approved P2781R7 (with design changes not relevant to this proposal, which are expressed as P2781R8) and forwarded it to LWG on 2025/03/11 for C++26. The only aspect of a type that can vary in the above list is `constant_wrapper`’s `Value`.

This solution would make it always ill-formed to call an existing submdspan_mapping customization with a new kind of slice. With the status quo, calling an existing submdspan_mapping customization with a new kind of slice would be ill-formed *at best*, and could be a precondition violation at worst, because nothing prevents submdspan from ever calling existing submdspan_mapping customizations with new slice types.

§ 5.2 Canonicalization rules

Here are the canonicalization rules for an input slice `s` of type `S` and a input mapping whose extents type has index type `index_type`. The result depends only on the input mapping’s extents type and the slices.

1. If `S` is convertible to `full_extent_t`, then `full_extent_t`;

2. else, if *S* is convertible to `index_type`, then `canonical-ice<index_type>(s)`;
3. else, if *S* is a specialization of `strided_slice`, then
`strided_slice{.offset=canonical-ice<index_type>(s.offset),
.extent=canonical-ice<index_type>(s.extent), .stride=canonical-
ice<index_type>(s.stride)};`
4. else, if *s* is destructurable into `[first, last]` that are both convertible to `index_type`, then `strided_slice{.offset=canonical-ice<index_type>(first), .extent=subtract-ice<index_type>(last, first), .stride=cw<index_type>(1)>}.`

The two exposition-only functions *canonical-ice* and *subtract-ice* preserve “compile-time-ness” of any *integral-constant-like* arguments, and force all inputs to either `index_type` or `constant_wrapper<Value, index_type>` for some `Value`. The *canonical-ice* function also has Mandates or Preconditions (depending on whether its argument is *integral-constant-like*) that its argument is representable as a value of type `index_type`. This lets implementations check for overflow. The `cw` variable template comes from [P2781](#).

§ 5.3 How to apply canonicalization

The function `submdspan_canonicalize_slices` canonicalizes all the slices at once. It takes as arguments the input mapping’s extents and the input slices of `submdspan_mapping`. Canonicalization makes `submdspan` equivalent to the following code.

```
auto [...canonical_slices] =
    submdspan_canonicalize_slices(src.extents(), slices...);
auto sub_map_result =
    submdspan_mapping(src.mapping(), canonical_slices...);
return mdspan(
    src.accessor().offset(src.data_handle(), sub_map_result.offset),
    sub_map_result.mapping,
    typename AccessorPolicy::offset_policy(src.accessor()));
```

The statement starting with `auto [...canonical_slices]` above uses the “structured bindings can introduce a pack” feature introduced by [P1061R10](#) into C++26. The `submdspan_canonicalize_slices` function takes the extents for two reasons.

1. A slice’s canonical type depends on the extents’ `index_type`.
2. Knowing the extents lets it enforce preconditions on the slices, so that the resulting canonical slices don’t need to be checked again.

§ 5.4 Canonicalization is reusable

We expose `submdspan_canonicalize_slices`, instead of making it exposition-only, because users may want to use it to build their own functionality analogous to `submdspan`. For example, consider a C++ type `small_matrix<T, M, N>` representing a dense rank-2 array with element type `T` and compile-time extents `M` and `N`. (This could be a library exposure of a compiler extension, such as [Clang’s matrix type extension](#).) This type would expose access to individual elements, but would *not* expose subviews. That is, getting an `mdspan` that views any subset of elements would be forbidden. However, `small_matrix` could expose *copies* of subsets of elements, via a function `submatrix` that takes the same slice specifiers as `submdspan`.

```
small_matrix<float, 5, 7> A = /* ... */;
auto A_sub = submatrix(A,
    strided_slice{.offset=1, .extent=4, .stride=3}, // 1, 4
    strided_slice{.offset=0, .extent=7, .stride=2}); // 0, 2, 4, 6
```

If `A` were a `layout_right` `mdspan`, then result of `submdspan` with those slices would be `layout_stride`. However, `A` is not an `mdspan`. Its layout might be hidden from users. Even if `small_matrix` layouts are known, there’s no need for `submatrix` to return layouts in the same way as `submdspan` does. For example, it might “pack” the result into a contiguous `layout_right` `small_matrix`. Nevertheless, the slice specifiers have the same interpretation for `submatrix` as for `submdspan`; they are independent of the input’s and result’s layout mappings. Thus, users who want to implement `submatrix` could use `submdspan_canonicalize_slices` to canonicalize `submatrix`’s slice arguments.

This example explains why `submdspan_mapping` has a name with the word “`submdspan`” in it, even though it is purely a function of layout mappings and need not be tied to `mdspan`. Calling `submdspan_mapping(mapping, slices...)` returns the mapping that `submdspan(input, slices...)` would have for `mapping = input.mapping()`.

§ 5.5 What about precondition checking?

The `submdspan` function imposes preconditions on both its slice arguments and on the result of `submdspan_mapping` ([\[mdspan.sub.sub\] 5](#)). The preconditions on `submdspan_mapping`’s result require that the user’s customization give a mapping that views exactly those elements of the input `mdspan` that are specified by the slices. That is, the customization has to be correct, given valid input slices. Enforcement of this is the user’s responsibility.

Enforcement of preconditions on the input slices is the implementation’s responsibility. This depends only on the input slices and the input mapping’s extents. Just as `mdspan` can check bounds for the input of the `at` function using the input and its extents, `submdspan` can enforce all preconditions on the slices using the input mapping’s extents. The current wording expresses these preconditions identically in two places:

1. on the slice arguments of `submdspan_extents` ([\[mdspan.sub.extents\] 3](#)), and
2. on the slice arguments of `submdspan` ([\[mdspan.sub.sub\] 5](#)).

Passing the slices through `submdspan_extents` is not enough to ensure that `submdspan_mapping`'s slice arguments do not violate its (unstated) preconditions. This is because deciding the result mapping's type and constructing it may depend on the slices, not just the result's extents. Another way to say this is that different `slices...` arguments might result in the same extents, but a different layout mapping. For example, if `src` is a rank-1 `layout_left` mapping with extents `dims<1>{10}`, both `full_extent` and `strided_slice{.offset=0, .extent=src.extent(0), .stride=1}` would result in extents `dims<1>{10}`, but different layouts (`layout_left` and `layout_stride`, respectively). This means that slice canonicalization is also an opportunity to avoid duplication of precondition-checking code.

There are two kinds of preconditions on slices:

1. that all indices represented by the slice can be represented as values of type `extents::index_type`, that is, that casting them to `extents::index_type` does not cause integer overflow; and
2. that all indices represented by slice `k` are in `[0, src.extent(k) )`.

The current wording expresses both preconditions via the exposition-only functions `first_` and `last_`. The specification of each of these uses `extents::index-cast` to preprocess their return values (see [\[mdspan.sub.helpers 4\]](#) and [\[mdspan.sub.helpers 9\]](#)). The `index-cast` exposition-only function [\[mdspan.extents.expo\] 9](#) is a hook to enforce the “no overflow” precondition. The “in the extent's range” precondition is currently up to `submdspan` implementations to check.

Slice canonicalization needs to have at least the “no overflow” precondition, because any integer results are either `index_type` or `constant_wrapper<Value, index_type>` for some `Value`. That is, it always casts input to `index_type`. We propose applying the “in the extent's range” precondition to slice canonicalization as well. This would let `submdspan` implementations put all their precondition checking (if any) in the canonicalization function. Users who implement a `submdspan_mapping` customization could thus assume that the input slices are all correct.

§ 5.6 Why implementers should do this anyway

Implementers should canonicalize slices anyway, even if this proposal is not adopted. Suppose, for example, that P2769 (expanding the definition of *tuple-like*) is adopted into some future C++ version. Implementations might thus expose users to an intermediate period in which they implement C++26 but not C++29. During this period, users might write `submdspan_mapping` customizations to the narrower definition of *pair-like*. For example, users might write `std::get` explicitly, which would only work with Standard Library types that have overloaded `std::get`, instead of using structured binding to get the two elements. This would, in turn, encourage implementers of `submdspan` to “canonicalize” *pair-like* slice arguments (e.g., into `pair` or `tuple`) before calling `submdspan_mapping` customizations, as that would maximize backwards compatibility.

In summary, the possibility of later generalization of *pair-like* would naturally lead high-quality implementations to the desired outcome. That would minimize code changes both for users, and for implementations (as the Standard Library currently has five `submdspan_mapping` customizations of its own). This proposal merely anticipates the expected implementation approach.

§ 6 Performance

§ 6.1 Speculation on compile-time and run-time effects

We show some preliminary performance results below, but they do not show a conclusive effect on performance. Thus, everything we write here is speculation.

In terms of compile-time performance, canonicalization would minimize the set of instantiations of both `submdspan_mapping` and `submdspan_extents` (which customizations of `submdspan_mapping` may call).

In terms of run-time performance, customizations would likely need to copy slices into their canonical forms. This would introduce new local variables. Slices tend to be simple, and are made of combinations of zero to three integers and empty types (like `constant_wrapper` or `full_extent_t`). Thus, it should be easy for compilers to optimize away all the extra types. On the other hand, if they can't, then the compiler may need to reserve extra hardware resources (like registers and stack space) for the extra local variables. This *may* affect performance, especially in tight loops. Mitigations generally entail creating subview layout mappings by hand.

The proposed set of canonicalization rules would require turning every *pair-like* type into a `strided_slice`. Pair-like slices are common enough in practice that we may want to optimize specially for them; see below. On the other hand, `strided_slice` is a simple aggregate type that might be easier for compilers to optimize. Users might thus prefer it over `pair` or `tuple`.

§ 6.2 Hypothetical performance mitigations

Here are some options that might mitigate some performance concerns. All such concerns are hypothetical without actual performance measurements.

1. Let Standard layout mappings accept arbitrary slice types. Restrict the proposed changes to user-defined layout mappings. This would optimize for the common case of Standard layout mappings. On the other hand, performing canonicalization for all `submdspan_mapping` customizations (including the Standard ones) would simplify implementations, especially for checking preconditions.
2. Expand the canonicalization rules so that `pair` and `tuple` slices are passed through, instead of being transformed into `strided_slice`. This would optimize a common case, at the cost of making `submdspan_mapping` customizations handle more slice

types. That would complicate the code and possibly increase compile-time cost, thus taking away at least some of the benefits of canonicalization.

§ 6.3 submdspan can skip canonicalization for Standard layout mappings

We believe that by the as-if rule, submdspan should be able to skip canonicalization for layout mappings provided by the Standard, even though we specify submdspan as “Effects: Equivalent to” code that always canonicalizes slices. If LWG disagrees, we offer alternate wording that enforces this permission.

§ 7 Implementation

[Pull Request 408](#) in the mdspan reference implementation’s [GitHub repository](#) implements this proposal.

§ 7.1 Configuration and build

The same branch can be used for testing with P3663 support either enabled or disabled. To enable P3663 support, set the CMake option MDSPAN_ENABLE_P3663 to ON.

The branch includes two different implementations of P3663 support. The default version requires C++20. The nondefault version requires the C++23 feature “deducing this,” and optionally can make use of the C++26 feature “[pack indexing](#)” (P2662R3) if available. The current development version of Clang (21.0.0) implements pack indexing; the latest release of GCC (15.1.0) does not. Thus, the code checks GCC version macros to decide whether to use the feature.

The default implementation has been tested with Clang 14 and GCC 11.4.0. The nondefault version has been tested with Clang 21.0.0 and GCC 15.1.0. To enable the default version, please use the following CMake options.

```
-DMDSPAN_ENABLE_P3663=ON \  
-DCMAKE_CXX_FLAGS="-std=c++20" \  
-DMDSPAN_CXX_STANDARD=20 \  
-DMDSPAN_CONSTANT_WRAPPER_WORKAROUND=ON
```

To enable the nondefault version, please use the following CMake options.

```
-DMDSPAN_ENABLE_P3663=ON \  
-DCMAKE_CXX_FLAGS="-std=c++2c" \  
-DMDSPAN_CXX_STANDARD=26 \  
-DMDSPAN_CONSTANT_WRAPPER_WORKAROUND=OFF
```

§ 7.2 Benchmark

§ 7.2.1 Benchmark code details

The P3663 branch includes benchmarks in the source file `benchmarks/submdspan/submdspan.cpp`. We show results for “host_benchmark2” below. This iterates many times over all the elements of a rank-6 mdspan of `uint8_t`, and multiplies each one in place by 3u. Knowing the initial elements (which are generated randomly) and the number of iterations lets us compute the expected results deterministically by repeated squaring modulo 256. Iteration over all the elements happens by recursive repeated slicing `submdspan(x, k, full_extent, ..., full_extent)` (where `full_extent, ..., full_extent` is not the actual syntax, but represents `rank() - 1` instances of `full_extent`). Slicing continues until the benchmark reaches a rank-1 mdspan, at which point the code just iterates over all the elements. Since the mdspan is `layout_right`, this iteration happens over contiguous data. We choose extents 4, 4, 4, 4, 4, 2 so that as much slicing as possible happens, while still keeping the working set in cache.

All slice arguments of `submdspan` are obfuscated in order to exercise the cost of slice conversions. The loop index `k` here is the mdspan’s `index_type`, but obfuscated by being stored in an `index_holder` wrapper type that is convertible to `index_type`. Likewise, `full_extent` in the expression above is actually `full_extent_wrapper_t{}`, an instance of an empty type that is convertible to `full_extent_t`.

The benchmarks shown here use the CMake flags and build configurations described above. Results generally use release builds (`CMAKE_BUILD_TYPE=Release`) without other CMake options that would change flags.

We show results for four different extents specializations.

1. `extents<int, 4, 4, 4, 4, 4, 2>` (rank 6, `IndexType=int`, all static extents)
2. `dims<6, int>{4, 4, 4, 4, 4, 2}` (rank 6, `IndexType=int`, all dynamic extents)
3. `extents<size_t, 4, 4, 4, 4, 4, 2>` (rank 6, `IndexType=size_t` (unsigned long), all static extents)
4. `dims<6, size_t>{4, 4, 4, 4, 4, 2}` (rank 6, `IndexType=size_t` (unsigned long), all dynamic extents)

The benchmark repeats iteration over all the elements of the mdspan 10^4 (10,000) times before a time measurement. We call this the “inner iteration count.” Google Benchmark then repeats this for a number of iterations that it decides – which we call the “outer iteration count” – before reporting a mean time over all outer iterations.

§ 7.2.2 Preliminary results

We show results with four different compilers.

1. Clang 21.0.0
2. GCC 15.1.0
3. Clang 14.0.0

4. GCC 11.4.0

For the first two compilers, we use the default version of the implementation, that assumes at least C++23. For the last two compilers, we use the nondefault “back-port” version that assumes only C++20. We include the back-port version because it would be how many users – including users of Kokkos, who rely on the reference mdspan implementation as the back-end for Kokkos::View – would experience P3663’s changes.

The following results were collected on an 11th Gen Intel(R) Core(TM) i7-11850H running at 2496 MHz. It reports 16 processors, with L1 and L2 caches shared between pairs of processors and the L3 cache shared over all processors. The L1 data cache is 48 KiB, the L1 instruction cache is 32 KiB, the L2 unified cache is 1280 KiB, and the L3 unified cache is 24576 KiB.

Benchmarks were run on Windows Subsystem Linux 2 (Ubuntu 22 LTS) under Windows 11. We show the “CPU” time as reported by Google Benchmark.

§ 7.2.2.1 extents<int, 4, 4, 4, 4, 4, 2> (rank 6, IndexType=int, all static extents)

Compiler	P3663 enabled?	Mean CPU time (ns)	Outer iterations
Clang 21.0.0	Yes	321917	2160
Clang 21.0.0	No	12612291	52
GCC 15.1.0	Yes	322847	2035
GCC 15.1.0	No	322098	2156
Clang 14.0.0	Yes	317226	2160
Clang 14.0.0	No	335550	2151
GCC 11.4.0	Yes	663204	1110
GCC 11.4.0	No	655524	1072

§ 7.2.2.2 dims<6, int>{4, 4, 4, 4, 4, 2} (rank 6, IndexType=int, all dynamic extents)

Compiler	P3663 enabled?	Mean CPU time (ns)	Outer iterations
Clang 21.0.0	Yes	28573409	25
Clang 21.0.0	No	32210084	22
GCC 15.1.0	Yes	24218781	29
GCC 15.1.0	No	21664452	31
Clang 14.0.0	Yes	16802927	42
Clang 14.0.0	No	13018827	55
GCC 11.4.0	Yes	29826258	25
GCC 11.4.0	No	23318382	30

§ 7.2.2.3 extents<size_t, 4, 4, 4, 4, 4, 2> (rank 6, IndexType=size_t, all static extents)

Compiler	P3663 enabled?	Mean CPU time (ns)	Outer iterations
Clang 21.0.0	Yes	323624	2099
Clang 21.0.0	No	9560465	22
GCC 15.1.0	Yes	322152	2123
GCC 15.1.0	No	318111	2191
Clang 14.0.0	Yes	326443	2193
Clang 14.0.0	No	9566053	72
GCC 11.4.0	Yes	679657	1044
GCC 11.4.0	No	673962	977

§ 7.2.2.4 `dims<6, size_t>{4, 4, 4, 4, 4, 2}` (rank 6, `IndexType=size_t`, all dynamic extents)

Compiler	P3663 enabled?	Mean CPU time (ns)	Outer iterations
Clang 21.0.0	Yes	11672089	57
Clang 21.0.0	No	29269825	25
GCC 15.1.0	Yes	25481788	27
GCC 15.1.0	No	22775149	32
Clang 14.0.0	Yes	28532969	25
Clang 14.0.0	No	14644212	45
GCC 11.4.0	Yes	21809043	31
GCC 11.4.0	No	26799987	26

§ 7.2.3 Preliminary conclusions

The only conclusion we can make from the above results is that P3663 would not obviously make the reference implementation of `mdspan` slower. Enabling P3663 support improves performance in some cases, especially for Clang 21.

Regardless of whether P3663 support is enabled, compilers vary in their ability to optimize `submdspan` calls. We see this both for different compilers, and for different versions of the same compiler. Optimization tends to be sensitive to code changes, even to not directly related changes like increasing or decreasing the number of lines of code in a file. Understanding why would take a closer analysis of the generated code.

§ 8 Acknowledgements

Thanks to Tomasz Kamiński for pointing out this issue and for suggesting improvements to the proposed wording.

§ 9 Proposed wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.

Assume that [P2781R8](#) has been applied to the Working Draft. [*Editorial note:* This makes `constant_wrapper` and `cw` available to the rest of the Standard Library. – *end note*]

§ 9.1 Increment `__cpp_lib_submdspan` feature test macro

In [version.syn], increase the value of the `__cpp_lib_submdspan` macro by replacing YYYYMM below with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_submdspan YYYYMM // also in <mdspan>
```

[*Editorial note:* If this proposal is adopted for C++26, increasing the version macro is not strictly necessary, because `submdspan` (P2630) itself is a C++26 feature. We retain the version macro increase in case the proposal is adopted after C++26, possibly as a DR for C++26. – *end note*]

§ 9.2 Change [mdspan.syn]

Change [mdspan.syn] (“Header `<mdspan>` synopsis”) as follows.

```
struct full_extent_t { explicit full_extent_t() = default; };
inline constexpr full_extent_t full_extent{};
```

```
template<class IndexType, size_t... Extents, class... Slices>
constexpr auto submdspan_canonicalize_slices(
    const extents<IndexType, Extents...>& src, Slices...
slices);
```

```
template<class IndexType, class... Extents, class...
SliceSpecifiers>
constexpr auto submdspan_extents(const extents<IndexType,
Extents...>&, SliceSpecifiers...);
```

§ 9.3 Change [mdspan.sub.helpers]

Change [mdspan.sub.helpers] as follows.

```
template<class T>
constexpr T de-ice(T val) { return val; }
template<integral-constant-like T>
constexpr auto de-ice(T) { return T::value; }
```

```
template<class IndexType, class S>
constexpr auto canonical-ice(S s);
```

¹ *Constraints:* `S` is convertible to `IndexType`.

2 *Mandates:*

- (2.1) — `IndexType` is a signed or unsigned integer type.
- (2.2) — If S models *integral-constant-like* and if `decltype(S::value)` is a signed or unsigned integer type, then $S::value$ is representable as a value of type `IndexType`.

3 *Preconditions:* If S is a signed or unsigned integer type, then s is representable as a value of type `IndexType`.

4 *Returns:*

- (4.1) — `cw<IndexType(extents<IndexType>::index_cast(S::value))>` if S models *integral-constant-like*;
- (4.2) — `IndexType(extents<IndexType>::index_cast(s))` otherwise.

```
template<class IndexType, class X, class Y>
constexpr auto subtract-ice(X x, Y y) {
    return canonical-ice<IndexType>(y) - canonical-ice<IndexType>
(x);
}
```

[*Editorial note:* Even though `submdspan` only uses the difference between y and x to compute an extent, each of y and x must be a valid index in the slice's extent. This is why *subtract-ice* retains *canonical-ice*'s precondition. – end note]

```
template<class IndexType, size_t k, class... SliceSpecifiers>
constexpr IndexType first_(SliceSpecifiers... slices);
```

[*Editorial note:* Former paragraph 1 has been renumbered to 5. – end note]

[*Editorial note:* `first_` only ever sees canonical `submdspan` slice types. – end note]

5 *Mandates:* ~~`IndexType` is a signed or unsigned integer type.~~

- (5.1) — `IndexType` is a signed or unsigned integer type.
- (5.2) — The declaration `auto [_, _] =  $s_k$` ; is well-formed if none of the following are true:
 - (5.2.1) — S_k is a canonical `submdspan` index type for `IndexType` (`[mdspan.sub.slices]`);
 - (5.2.3) — S_k is `full_extent_t`;
 - (5.2.4) — S_k is a specialization of `strided_slice` where all of the following are true:
 - (5.2.4.1) — $S_k::offset_type$, $S_k::extent_type$, and $S_k::stride_type$ are all canonical `submdspan` index types for `IndexType`; and
 - (5.2.4.2) — if $S::stride_type$ is `constant_wrapper<Stride, IndexType>` for some value `Stride` of type `IndexType`, and if $S::extent_type$ is

constant_wrapper<Extent, IndexType> for some value Extent of type IndexType, then either Extent equals zero, or Stride is greater than zero.

[*Editorial note*: 5.2 effectively repeats the definition of “canonical submdspan slice type in [mdspan.sub.slices] 2. It would be interesting to find a more concise way to express 5.2, since we need to say things like this elsewhere. Perhaps we could add a definition like “canonical submdspan non-pair-like slice type” to [mdspan.sub.slices]. On the other hand, this could induce confusion with the exposition-only concept *index-pair-like*, which is an opt-in list of types and does not use the “structured binding into two elements is well-formed” definition. – end note]

6 Let ϕ_k denote the following value:

- (6.1) — s_k if S_k ~~models convertible_to<IndexType>;~~ is a canonical submdspan index type for IndexType ([mdspan.sub.slices]); otherwise

[*Editorial note*: The change to 6.1 is not strictly necessary, but it emphasizes that *first_* only ever sees canonical submdspan slice types. – end note]

- (6.2) — ~~otherwise, get<0>(s_k) if S_k models *index-pair-like*<IndexType>;~~
- (6.3) — ~~otherwise, de-ice(s_k .offset) if S_k is a specialization of *strided_slice*;~~
- (6.4) — ~~otherwise, 0.~~
- (6.2) — 0 if S_k is *full_extent_t*; otherwise
- (6.3) — ~~de-ice(s_k .offset) if S_k is a specialization of *strided_slice*; otherwise~~
- (6.4) — s_{k0} produced by `auto [sk0, _] = sk;`

[*Editorial note*: We reordered the bullet list in the definition of ϕ_k so that the “structured binding into two elements is well-formed” case is last in the list. We consider this a reasonable way to implement the Mandates. – end note]

7 *Preconditions*: ϕ_k is representable as a value of type IndexType.

8 *Returns*: `extents<IndexType>::index-cast( $\phi_k$ )`.

```
template<size_t k, class Extents, class... SliceSpecifiers>
constexpr auto last_(const Extents& src, SliceSpecifiers...
slices);
```

9 *Mandates*: ~~Extents is a specialization of extents.~~

- (9.1) — Extents is a specialization of extents.
- (9.2) — The declaration `auto [_, _] = sk;` is well-formed if none of the following are true:
- (9.2.1) — S_k is a canonical submdspan index type for IndexType ([mdspan.sub.slices]);

- (9.2.3) — S_k is `full_extent_t`;
- (9.2.4) — S_k is a specialization of `strided_slice` where all of the following are true:
 - (9.2.4.1) — $S_k::\text{offset_type}$, $S_k::\text{extent_type}$, and $S_k::\text{stride_type}$ are all canonical submdspan index types for `IndexType`; and
 - (9.2.4.2) — if $S::\text{stride_type}$ is `constant_wrapper<Stride, IndexType>` for some value `Stride` of type `IndexType`, and if $S::\text{extent_type}$ is `constant_wrapper<Extent, IndexType>` for some value `Extent` of type `IndexType`, then either `Extent` equals zero, or `Stride` is greater than zero.

[*Editorial note:* For 9.2, please see Editorial Note on *first_*. – *end note*]

10 Let `index_type` be typename `Extents::index_type`.

11 Let λ_k denote the following value:

- (11.1) — `de-ice( $s_k$ ) + 1` if S_k ~~models convertible_to<index_type>~~ is a canonical submdspan index type for index_type ([mdspan.sub.slices]); otherwise

[*Editorial note:* The change to 11.1 is not strictly necessary, but it emphasizes that *last_* only ever sees canonical submdspan slice types. – *end note*]

- (11.2) — `get<1>( $s_k$ )` if S_k models *index-pair-like*<index_type>; otherwise
- (11.3) — `de-ice( $s_k$ .offset) + de-ice( $s_k$ .extent)` if S_k is a specialization of `strided_slice`; otherwise
- (11.4) — `src.extent(k)`.
- (11.2) — `src.extent(k)` if S_k is `full_extent_t`; otherwise
- (11.3) — `de-ice( $s_k$ .offset) + de-ice( $s_k$ .extent)` if S_k is a specialization of `strided_slice`; otherwise
- (11.4) — `s_k1` produced by `auto [_, s_k1] = s_k;`.

12 *Preconditions:* λ_k is representable as a value of type `index_type`.

13 *Returns:* `Extents::index-cast( $\lambda_k$ )`.

```
enum class check-static-bounds-result {
    in-bounds,
    out-of-bounds,
    unknown
};

template<size_t k, class Slice, class IndexType, size_t... Exts>
constexpr check-static-bounds-result check-static-bounds(
    const extents<IndexType, Exts...>&);
```

14 *Returns:*

- (14.1) — if `is_convertible_v<Slice, full_extent_t>` is true, *check-static-bounds-result::in-bounds*;
- (14.2) — otherwise, if `Slice` is *integral-constant-like* and `is_convertible_v<Slice, IndexType>` is true:
 - (14.2.1) — if `de-ice(Slice{}) < 0`, *check-static-bounds-result::out-of-bounds*;
 - (14.2.2) — otherwise, if `Exts...[k]` does not equal `dynamic_extent` and if `Exts...[k] ≤ de-ice(Slice{})`, *check-static-bounds-result::out-of-bounds*;
 - (14.2.3) — otherwise, if `Exts...[k]` does not equal `dynamic_extent` and if `de-ice(Slice{}) < Exts...[k]`, *check-static-bounds-result::in-bounds*;
 - (14.2.4) — otherwise, *check-static-bounds-result::unknown*;
- (14.3) — otherwise, if `Slice` is a specialization of `strided_slice` and each of `Slice::offset_type`, `Slice::extent_type`, and `Slice::stride_type` are convertible to `IndexType`:
 - (14.3.1) — if `Slice::offset_type` models *integral-constant-like*:
 - (14.3.1.1) — if `de-ice(Slice::offset_type{}) < 0`, *check-static-bounds-result::out-of-bounds*;
 - (14.3.1.2) — otherwise, if `Exts...[k]` does not equal `dynamic_extent` and if `Exts...[k] < de-ice(Slice::offset_type{})`, *check-static-bounds-result::out-of-bounds*;
 - (14.3.1.3) — otherwise, if `Slice::extent_type` models *integral-constant-like* and if `de-ice(Slice::offset_type{}) + de-ice(Slice::extent_type{}) < 0`, *check-static-bounds-result::out-of-bounds*;
 - (14.3.1.4) — otherwise, if `Exts...[k]` does not equal `dynamic_extent`, if `Slice::extent_type` models *integral-constant-like*, and if `Exts...[k] < de-ice(Slice::offset_type{}) + de-ice(Slice::extent_type{})`, *check-static-bounds-result::out-of-bounds*;
 - (14.3.1.5) — otherwise, if `Exts...[k]` does not equal `dynamic_extent`, if `Slice::extent_type` models *integral-constant-like*, and if `0 ≤ de-ice(Slice::offset_type{}) ≤ de-ice(Slice::offset_type{}) + de-ice(Slice::extent_type{}) ≤ Exts...[k]`, *check-static-bounds-result::in-bounds*;
 - (14.3.1.6) — otherwise, *check-static-bounds-result::unknown*;
 - (14.3.2) — otherwise, *check-static-bounds-result::unknown*;

- (14.4) — otherwise, let `s_k0` and `s_k1` be produced by `auto [s_k0, s_k1] = declval<Slice>();`, let `S_k0` be `decltype(s_k0)`, and let `S_k1` be `decltype(s_k1)`:
 - (14.4.1) — if `S_k0` is *integral-constant-like*:
 - (14.4.1.1) — if `de-ice(S_k0{}) < 0`, `check-static-bounds-result::out-of-bounds`;
 - (14.4.1.2) — otherwise, if `Exts... [k]` does not equal `dynamic_extent` and if `Exts... [k] < de-ice(S_k0{})`, `check-static-bounds-result::out-of-bounds`;
 - (14.4.1.3) — otherwise, if `S_k1` is *integral-constant-like* and if `de-ice(S_k1{}) < de-ice(S_k0{})`, `check-static-bounds-result::out-of-bounds`;
 - (14.4.1.4) — otherwise, if `Exts... [k]` does not equal `dynamic_extent`, if `S_k1` is *integral-constant-like*, and if `Exts... [k] < de-ice(S_k1{})`, `check-static-bounds-result::out-of-bounds`;
 - (14.4.1.5) — otherwise, if `Exts... [k]` does not equal `dynamic_extent`, if `S_k1` is *integral-constant-like*, and if `0 ≤ de-ice(S_k0{}) ≤ de-ice(S_k1{}) ≤ Exts... [k]`, `check-static-bounds-result::in-bounds`;
 - (14.4.1.6) — otherwise, `check-static-bounds-result::unknown`;
 - (14.4.2) — otherwise, `check-static-bounds-result::unknown`.

[*Editorial Note:* The intent of 14.4 is that if none of 14.1, 14.2, or 14.3 hold, then 14.4 effectively implements a Mandate that `auto [s_k0, s_k1] = declval<Slice>();` be well-formed. We implemented this using an `if constexpr` (14.1 condition) ... `else if constexpr` (14.2 condition) ... `else if constexpr` (14.3 condition) ... `else` chain. Note that the 14.4 case should not require that `Slice` be default-constructible. In case the `constexpr`-ability of this is not clear, here is how we implemented getting `decltype(s_k0)` and `decltype(s_k1)` in the final `else` branch.

```
auto get_first = [] (Slice s_k) {
    auto [s_k0, _] = s_k;
    return s_k0;
};
auto get_second = [] (Slice s_k) {
    auto [_, s_k1] = s_k;
    return s_k1;
};
using S_k0 = decltype(get_first(std::declval<Slice>()));
using S_k1 = decltype(get_second(std::declval<Slice>()));
```

– *end note*

```
template<class IndexType, size_t N, class... SliceSpecifiers>
constexpr array<IndexType, sizeof...(SliceSpecifiers)>
    src-indices(const array<IndexType, N>& indices,
SliceSpecifiers... slices);
```


16 *Returns:* An array<IndexType, sizeof...(SliceSpecifiers)> src_idx such that for each k in the range $[0, \text{sizeof} \dots (\text{SliceSpecifiers}))$, src_idx[k] equals

(16.1) — $\text{first_}<\text{IndexType}, k>(\text{slices} \dots)$ for each k where $\text{map-rank}[k]$ equals dynamic_extent ,

(16.2) — otherwise, $\text{first_}<\text{IndexType}, k>(\text{slices} \dots) + \text{indices}[\text{map-rank}[k]]$.

§ 9.4 Add section [mdspan.sub.slices], “Slices definitions and canonicalization”

Add a new section [mdspan.sub.slices], “Slices definitions and canonicalization,” after [mdspan.sub.helpers] and before [mdspan.sub.extents]. The new section has the following contents.

1 Given a signed or unsigned integer IndexType, a type S is a *canonical submdspan index type for IndexType* if S is either IndexType or $\text{constant_wrapper}<\text{Value}, \text{IndexType}>$ for some Value of type IndexType.

2 Given a signed or unsigned integer IndexType, a type S is a *canonical submdspan slice type for IndexType* if exactly one of the following is true:

(2.1) — S is full_extent_t ;

(2.2) — S is a canonical submdspan index type for IndexType; or

(2.3) — S is a specialization of strided_slice where all of the following are true:

(2.3.1) — $S::\text{offset_type}$, $S::\text{extent_type}$, and $S::\text{stride_type}$ are all canonical submdspan index types for IndexType; and

(2.3.2) — if $S::\text{stride_type}$ is $\text{constant_wrapper}<\text{Stride}, \text{IndexType}>$ for some value Stride of type IndexType, and if $S::\text{extent_type}$ is $\text{constant_wrapper}<\text{Extent}, \text{IndexType}>$ for some value Extent of type IndexType, then either Extent equals zero, or Stride is greater than zero.

3 Given an extents specialization E and a rank index k of E , a type S_k is a *canonical k^{th} submdspan slice type for E* if all of the following are true:

(3.1) — S_k is a canonical submdspan slice type for $E::\text{index_type}$, and

(3.2) — $\text{check-static-bounds}<k, S_k>(E\{\})$ does not equal $\text{check-static-bounds-result}::\text{out-of-bounds}$.

4 Given a signed or unsigned integer IndexType, a type S is a *valid submdspan slice type for IndexType* if exactly one of the following is true:

(4.1) — $\text{is_convertible_v}<S, \text{full_extent_t}>$ is true;

- (4.2) — `is_convertible_v<S, IndexType>` is true;
- (4.3) — `S` is a specialization of `strided_slice`; or
- (4.4) — if `s` is an object of type `S`, then the structured binding declaration `auto [s0, s1] =s;` is well-formed, and each of `s0` and `s1` are convertible to `IndexType`.

[*Note 1:* If `S` is a valid submdspan slice type for `IndexType`, then it is also a canonical slice type for `IndexType`. — *end note*]

5 Given an extents specialization `E` and a rank index k of `E`, a type S_k is a *valid k^{th} submdspan slice type for `E`* if all of the following are true:

- (5.1) — S_k is a valid submdspan slice type for `E::index_type`, and
- (5.2) — `check-static-bounds<k, Sk>(E{})` does not equal `check-static-bounds-result::out-of-bounds`.

[*Note 2:* If S_k is a valid k^{th} submdspan slice type for `E`, then it is also a canonical k^{th} submdspan slice type for `E`. — *end note*]

6 Given an extents specialization `E`, an object `e` of type `E`, and a rank index k of `E`, an object s_k of type S_k is a *valid k^{th} submdspan slice for `e`* if S_k is a valid k^{th} submdspan slice type for `E`, and if all of the following are true:

- (6.1) — If S_k is a specialization of `strided_slice`, then either
 - (6.1.1) — `sk.extent = 0`, or
 - (6.1.2) — `sk.stride > 0`; and
- (6.2) — `0 ≤ first_<IndexType, k>(slices...) ≤ last_<k>(src, slices...) ≤ src.extent(k)`.

7 Given an extents specialization `E`, an object `e` of type `E`, and a rank index k of `E`, an object s_k of type S_k is a *canonical k^{th} submdspan slice for `e`* if s_k is a valid k^{th} submdspan slice for `E`, and if S_k is a canonical k^{th} submdspan slice type for `E`.

```
template<class IndexType, size_t... Extents, class... Slices>
constexpr auto submdspan_canonicalize_slices(
    const extents<IndexType, Extents...>& src, Slices... slices);
```

8 *Constraints:* `sizeof...(slices)` equals `sizeof...(Extents)`.

9 *Mandates:* For each rank index k of `src`, S_k is a valid k^{th} submdspan slice type for `extents<IndexType, Extents...>`.

[*Editorial Note:* This cannot be a Constraint because it is impossible to constrain a template on whether a structured binding declaration is well-formed. This is because a compound requirement accepts an expression, not a declaration. – *end note*]

10 *Preconditions:* For each rank index k of `src`, s_k is a valid k^{th} submdspan slice for `src`.

11 *Returns:* a tuple of `src.rank()` elements, where for each rank index k of `src`, the following specifies element k of the return value:

- (11.1) — If `is_convertible_v< $S_k$ , full_extent_t>` is true, then `full_extent`;
- (11.2) — otherwise, if `is_convertible_v< $S_k$ , IndexType>` is true, then `canonical-ice<IndexType>(s)`;
- (11.3) — otherwise, if S_k is a specialization of `strided_slice`, then
`strided_slice{.offset=canonical-ice<IndexType>(s.offset),
.extent=canonical-ice<IndexType>(s.extent), .stride=canonical-ice<IndexType>(s.stride)};`
- (11.4) — otherwise, `strided_slice{.offset=canonical-ice<IndexType>(s_k0),
.extent=subtract-ice<IndexType>(s_k0, s_k1),
.stride=cw<IndexType(1)>}`, where `s_k0` and `s_k1` are produced by `auto [s_k0, s_k1] = s_k;`.

[*Note 3:* For each rank index k of `src`, element k of the tuple returned by `submdspan_canonicalize_slices` is a canonical k^{th} submdspan slice for `src`. – *end note*]

§ 9.5 Change section [mdspan.sub.extents]

Change section [mdspan.sub.extents] as follows.

```
template<class IndexType, class... Extents, class...
SliceSpecifiers>
constexpr auto submdspan_extents(const extents<IndexType,
Extents...>& src,
                                SliceSpecifiers... slices);
```

1 *Constraints:* `sizeof...(Slices)` equals `Extents::rank()`.

2 *Mandates:* For each rank index k of `src.extents()`, ~~exactly one of the following is true:~~ S_k is a valid k^{th} submdspan slice type for `extents<IndexType, Extents...>`.

- (2.1) — ~~S_k models convertible_to<IndexType>;~~
- (2.2) — ~~S_k models index-pair-like<IndexType>;~~
- (2.3) — ~~`is_convertible_v< $S_k$ , full_extent_t>` is true, or~~

(2.4) — ~~S_k is a specialization of `strided_slice`.~~

3 *Preconditions:* For each rank index k of `src`, ~~all of the following are true:~~ s_k is a valid k^{th} `submdspan` slice for `src`.

(3.1) — ~~If S_k is a specialization of `strided_slice`~~

(3.1.1) — ~~s_k .`extent` = 0, or~~

(3.1.2) — ~~s_k .`stride` > 0~~

(3.2) — ~~$0 \leq \text{first_}<\text{IndexType}, k>(\text{slices} \dots) \leq \text{last_}<k>(\text{src}, \text{slices} \dots) \leq \text{src.extent}(k)$~~

4 Let `SubExtents` be a specialization of `extents` such that:

§ 9.6 Requirements of all `submdspan_mapping` customizations

Right before `[mdspan.sub.map.common]` (“Specializations of `submdspan_mapping`”), insert a new section `[mdspan.sub.sliceable]`, “Sliceable layout mapping requirements,” with the following content.

```
template<layout-mapping-alike LayoutMapping>
constexpr auto
submdspan-mapping-with-full-extents(const LayoutMapping& mapping);
```

1 *Constraints:* `LayoutMapping` meets the layout mapping requirements (`[mdspan.layout.reqmts]`).

2 *Returns:* `submdspan_mapping(mapping, P)`, where P is a pack of typename `LayoutMapping::extents_type::rank()` object(s) of type `full_extent_t`.

[*Note 1:* This invocation of `submdspan_mapping` selects a function call via overload resolution on a candidate set that includes the lookup set found by argument-dependent lookup (`[basic.lookup.argdep]`). — *end note*]

```
template<class T>
constexpr bool is-submdspan-mapping-result = false;

template<class LayoutMapping>
constexpr bool is-submdspan-mapping-result<
    submdspan_mapping_result<LayoutMapping>> = true;

template<class LayoutMapping>
concept submdspan-mapping-result =
    is-submdspan-mapping-result<LayoutMapping>;

template<class LayoutMapping>
concept mapping-sliceable-with-full-extents =
    requires(const LayoutMapping& mapping) {
```

```

    {
        submdspan-mapping-with-full-extents(mapping)
    } -> submdspan-mapping-result;
};

```

- 3 A type T models *mapping-sliceable-with-full-extents* only if T is an object type that meets the layout mapping requirements ([mdspan.layout.reqmts]).

[Editorial Note: The exposition-only concept *mapping-sliceable-with-full-extents* exists because it is impossible to constrain `submdspan` ([mdspan.sub.sub]) on whether `submdspan_mapping(src.mapping(), slices...)` is well-formed. This is because it is impossible to constrain a function template on whether one of its arguments (a member of the `SliceSpecifiers` pack, in this case) can be on the right-hand side of a structured binding declaration with two elements. This, in turn, is because a structured binding declaration is a declaration, but constraints cannot use declarations, only expressions. — *end note*]

```

    template<layout-mapping-alike LayoutMapping, class... Slices>
    constexpr auto
    invoke-submdspan-mapping(const LayoutMapping& mapping, Slices...
slices) {
        return submdspan_mapping(mapping, slices...);
    }

```

- 4 Let M be a type, and let m be an object of type M . M meets the *sliceable layout mapping* requirements if all of the following are true.

- (4.1) — M models *mapping-sliceable-with-full-extents*.
- (4.2) — the expression `invoke-submdspan-mapping(mapping, slices...)` is well-formed only if `sizeof...(slices)` equals `LayoutMapping::extent_type::rank()`.

[Editorial Note: This forms the Constraints on any `submdspan_mapping` customization. — *end note*]

- (4.3) — Let `sizeof...(Slices)` equal `LayoutMapping::extents_type::rank()`, and for every rank index k of `LayoutMapping::extents_type`, let `Slices...[k]` be a canonical k^{th} `submdspan` slice type for `LayoutMapping::extents_type`. Then,
- (4.3.1) — the expression `invoke-submdspan-mapping(mapping, slices...)` is well-formed; and
- (4.3.2) — `decltype(invoke-submdspan-mapping(mapping, slices...))` is a specialization of `submdspan_mapping_result`.
- (4.4) — Let `sizeof...(Slices)` equal `LayoutMapping::extents_type::rank()`. If there exists a k in $[0, \text{sizeof...(Slices)})$ such that `Slices...[k]` is not a canonical k^{th} `submdspan` slice type for `LayoutMapping::extents_type`, then the expression `invoke-submdspan-mapping(mapping, slices...)` is ill-formed.

[*Note 2: Restricting submdspan_mapping customizations to accept only canonical slice types makes it possible for the set of valid slice types to grow in the future. — end note*]

[*Editorial Note: It is our belief that this wording and the as-if rule together permit Standard layout mappings to accept any valid submdspan slice types, and thus permit submdspan not to canonicalize for Standard layout mappings. Here is alternate wording if we need a special call-out for Standard layout mappings.*

(4.4) — Let `sizeof...(Slices)` equal `LayoutMapping::extents_type::rank()`.

(4.4.1) — If `LayoutMapping::layout_type` is a layout defined in this Standard, then if there exists a rank index k of `LayoutMapping::extents_type` such that `Slices...[k]` is not a valid k^{th} submdspan slice type for `LayoutMapping::extents_type`, then the expression `invoke-submdspan-mapping(mapping, slices...)` is ill-formed.

(4.4.2) — Otherwise, if there exists a rank index k of `LayoutMapping::extents_type` such that `Slices...[k]` is not a canonical k^{th} submdspan slice type for `LayoutMapping::extents_type`, then the expression `invoke-submdspan-mapping(mapping, slices...)` is ill-formed.

— *end note*]

[*Editorial Note: The above two points form the Mandates on any submdspan_mapping customization. — end note*]

(4.5) — Let `sizeof...(Slices)` equal `LayoutMapping::extents_type::rank()`, and for every rank index k of `LayoutMapping::extents_type`, let `slices...[k]` be a canonical k^{th} submdspan slice for `mapping.extents()`. Let `sub_map_offset` be the result of `invoke-submdspan-mapping(mapping, slices...)`. Then,

(4.5.1) — `sub_map_offset.mapping.extents() == submdspan_extents(src.mapping(), slices...)` is true; and

(4.5.2) — for each integer pack `I` which is a multidimensional index in `sub_map_offset.mapping.extents()`,

```
sub_map_offset.mapping(I...) + sub_map_offset.offset ==  
src.mapping()(src-indices(array{I...}, slices...))
```

is true.

[*Editorial Note: This forms the Effects of submdspan_mapping, and the Preconditions on using any submdspan_mapping customization in submdspan ([mdspan.sub.sub]). — end note*]

§ 9.7 Change [mdspan.sub.map.common]

Change [mdspan.sub.map.common] as follows.

1 The following elements apply to all functions in [mdspan.sub.map].

2 *Constraints:* sizeof...(slices) equals extents_type::rank().

3 *Mandates:* For each rank index k of extents(), S_k is a valid k^{th} submdspan slice type for extents_type.~~exactly one of the following is true:~~

(3.1) — ~~S_k models convertible_to<index_type>;~~

(3.2) — ~~S_k models index_pair_like<index_type>;~~

(3.3) — ~~is_convertible_v< S_k , full_extent_t> is true, or~~

(3.4) — ~~S_k is a specialization of strided_slice.~~

4 *Preconditions:* For each rank index k of extents(), S_k is a valid k^{th} submdspan slice for extents().~~all of the following are true:~~

(4.1) — ~~if S_k is a specialization of strided_slice, s_k .extent is equal to zero or s_k .stride is greater than zero; and~~

(4.2) — ~~$0 \leq \text{first_}<\text{index_type}, k>(\text{slices} \dots)$
 $0 \leq \text{last_}<k>(\text{extents}(), \text{slices} \dots)$
 $0 \leq \text{extents}().\text{extent}(k)$~~

5 Let sub_ext be the result of submdspan_extents(extents(), slices...) and let SubExtents be decltype(sub_ext).

§ 9.8 submdspan function template

Change [mdspan.sub.sub] (“submdspan function template”) as follows.

1 Let index_type be typename Extents::index_type.

2 Let canonical_slices be the pack resulting from the following statement.

```
auto [...canonical_slices] =  
submdspan_canonicalize_slices(src.extents(), slices...).
```

3 Let sub_map_offset be the result of submdspan_mapping(src.mapping(),
~~slices~~canonical_slices ...).

[Note 1: This invocation of submdspan_mapping selects a function call via overload resolution on a candidate set that includes the lookup set found by argument-dependent lookup ([basic.lookup.argdep]). — end note]

4 **M** shall meet the sliceable layout mapping requirements.

[*Editorial Note:* (4) cannot be fully implemented as a Constraint. The syntactic requirements have the effect of Mandates. The semantic requirements have the effect of Preconditions. — *end note*]

5 *Constraints:*

- (5.1) — `sizeof...(slices)` equals `Extents::rank()`, and
- (5.2) — ~~the expression `submdspan_mapping(src.mapping(), slices...)` is well-formed when treated as an unevaluated operand~~ `Layout::mapping<Extents>` meets the requirements of *mapping-sliceable-with-full-extents*.

6 *Mandates:*

- (6.1) — `decltype(submdspan_mapping(src.mapping(), slicescanonical slices...))` is a specialization of `submdspan_mapping_result`.
- (6.2) — `is_same_v<remove_cvref_t<decltype(sub_map_offset.mapping.extents())>, decltype(submdspan_extents(src.mapping(), slices...))>` is true.
- (6.1) — For each rank index k of `src.extents()`, ~~exactly one of the following is true:~~ S_k is a valid k^{th} submdspan slice type for `Extents`.
 - (6.1.1) — ~~S_k models convertible_to<index_type>.~~
 - (6.1.2) — ~~S_k models index-pair-like<index_type>.~~
 - (6.1.3) — ~~`is_convertible_v< $S_k$ , full_extent_t>` is true, or~~
 - (6.1.4) — ~~S_k is a specialization of `strided_slice`.~~

7 *Preconditions:*

- (7.1) — For each rank index k of `src.extents()`, s_k is a valid k^{th} submdspan slice for `src.extents()`. ~~all of the following are true:~~
 - (7.1.1) — ~~if S_k is a specialization of `strided_slice`~~
 - (7.1.1.1) — ~~`sk.extent` == 0, or~~
 - (7.1.1.2) — ~~`sk.stride` > 0~~
 - (7.1.2) — ~~`0 ≤ first<index_type, k>(slices) ≤ last<k>(src.extents(), slices...)` ≤ `src.extent(k)`~~
- (7.2) — `sub_map_offset.mapping.extents() == submdspan_extents(src.mapping(), slices...)` is true; and
- (7.3) — for each integer pack `I` which is a multidimensional index in `sub_map_offset.mapping.extents()`,


```
sub_map_offset.mapping(I...) + sub_map_offset.offset ==
src.mapping()(src-indices(array{I...}, slices...))
```

is true.

8 *Effects*: Equivalent to:

```
auto [...canonical_slices] =
submdspan_canonicalize_slices(src.extents(), slices...);
```

```
auto sub_map_result =
submdspan_mapping(src.mapping(), canonical_slices...);
return mdspan(
src.accessor().offset(src.data_handle(), sub_map_result.offset),
sub_map_result.mapping,
typename AccessorPolicy::offset_policy(src.accessor()));
```

[*Editorial note*: Please note drive-by fix for `src.data_handle()` (original was `src.data()`). – *end note*]

[*Editorial note*: Please note drive-by fix adding missing `typename` before `AccessorPolicy::offset_policy`. – *end note*]

[*Editorial note*: We believe that the above wording permits, but does not require canonicalization, for Standard layout mappings. If LWG does not agree, we offer the following alternative wording.

- (8.1) — If Layout is one of the layouts defined in this Standard and `Layout::mapping<Extents>` meets *mapping-sliceable-with-full-extents*, then equivalent to:

```
auto sub_map_result = submdspan_mapping(src.mapping(), slices...);
return mdspan(
src.accessor().offset(src.data_handle(), sub_map_result.offset),
sub_map_result.mapping,
typename AccessorPolicy::offset_policy(src.accessor()));
```

- (8.2) — Otherwise, ~~E~~Equivalent to:

```
auto [...canonical_slices] =
submdspan_canonicalize_slices(src.extents(), slices...);
```

```
auto sub_map_result =
submdspan_mapping(src.mapping(), canonical_slices...);
return mdspan(
src.accessor().offset(src.data_handle(), sub_map_result.offset),
sub_map_result.mapping,
typename AccessorPolicy::offset_policy(src.accessor()));
```

– *end note*]